

Content Disarm and Reconstruction as a Pre-Parse Defense for the DICOM File Attack Surface: A Differential-Evaluation Methodology

Vijay Thakore*

Draft preprint. Results reproduced 2026-05-25.

Open source (Apache-2.0): <https://github.com/vthakore23/dicomlock> Package: `pip install dicomlock`

Abstract

Medical images move through hospitals as files, and a DICOM file is not a picture but a parser’s input: a 128-byte preamble that can carry an executable header, attacker-controlled length and nesting fields, and pixel data that decodes through image and video codecs with long histories of memory-safety defects. AI-assisted vulnerability discovery now finds such defects faster than healthcare software can be recertified and patched, and the health sector was not among the organizations given early-access protection when one such system was announced. We present DicomLock, an open-source, self-hosted tool that scans DICOM files for the ways they can be weaponized and then disarms recoverable files through Content Disarm and Reconstruction (CDR): it zeroes the preamble, transcodes compressed pixel data off the vulnerable codec in a resource-limited sandbox, filters private tags against a default-deny vendor allowlist, and re-scans the rebuilt file, quarantining anything still dangerous. Alongside the artifact, we contribute the differential-evaluation methodology used to evaluate it: a labeled corpus of inert attacks is run through both a matrix of three production DICOM toolkits and through CDR, McNemar’s paired test is applied to the discordant cell, and the corpus is grown adversarially until it breaks the defense itself. The methodology is general to file-format CDR; we instantiate it on DICOM and publish the falsification report it produced, including a defect we found in our own engine. On the current corpus DicomLock detects 80 of 80 tampered files, produces 0 false positives across 605 benign files (575 real clinical CTs plus 30 curated) and across 370 further real clinical files in three additional body regions and modalities (100 abdomen CT, 120 brain MR, 150 chest radiographs), neutralizes 80 of 80 dangerous inputs, and rebuilds every native and lossless file bit-exact (623 in the fidelity harness plus the 370 additional files) across 13 transfer syntaxes. Single-threaded on commodity hardware, the scanner runs at 254 files per second across the 945-file corpus (peak 265 MiB), and the CDR rebuild runs at 63 to 174 files per second depending on whether the codec sandbox is invoked. Against the three reference toolkits (pydicom, GDCM, dcmtk), DicomLock flags 51 files that every toolkit accepts as valid, and no file it passes as clean is rejected by a toolkit (McNemar $\chi^2 = 49.0$, $p < 10^{-6}$). The adversarial round found and fixed a real defect in our own CDR, in which a payload hidden under an allowlisted vendor creator survived disarm; we report it as a worked example of the method. As a secondary contribution, we audit residual re-identification risk across 1,045 public “de-identified” files across five datasets and find substantial pixel-domain risk (facial-geometry features on 96.7 percent of head MR, burned-in text on 89.3 percent of chest radiographs and 52.0 percent of mammograms) that current tag anonymization is structurally unable to fix.

*Correspondence: vthakore@uchicago.edu. Affiliation to be completed at submission.

DicomLock is positioned as a compensating control for the systems an automated patch loop cannot reach, not as a replacement for patching.

1 Introduction

The bottleneck in software security has shifted from finding vulnerabilities to fixing them. Public reporting on AI-assisted vulnerability discovery in 2026 (Anthropic, Project Glasswing / Claude Mythos) [1, 2, 3] describes a model that found and exploited large numbers of high and critical defects in a short window, with the great majority left unpatched at disclosure; verifiable examples include 271 Firefox vulnerabilities surfaced in a single pass and a long-lived OpenBSD defect. (A widely cited 17-year-old FreeBSD NFS finding remains contested, with some researchers arguing it was recalled from training data rather than discovered fresh; we cite it only as “found and exploited via Mythos” and do not treat it as a settled novel discovery.) Secondary reporting describes the median time from disclosure to weaponization falling from hundreds of days to hours; we cite that figure as reported, not as a primary measurement.

Hospitals are among the slowest patchers, constrained by device recertification and by legacy and embedded equipment, and they run exactly the software in question: DICOM toolkits and image and video codecs. A 2026 advisory from Health-ISAC, reported alongside coverage in STAT, noted that the early access group for Glasswing-style protection did not include hospitals, device makers, or other health-sector entities, and warned that the omission could jeopardize health-sector security [4, 5]. That is the most concrete present-tense reason to harden medical imaging now, rather than an anticipatory one.

The DICOM file itself is the attack surface. The 128-byte preamble can carry an executable header, making one file simultaneously a valid scan and a valid executable (CVE-2019-11687 [6]; the ELFDICOM work [7] extended the technique to Linux medical devices). Element length fields are attacker-controlled, so a small file can declare an allocation large enough to crash a naive parser, and deeply nested sequences can exhaust memory or stack. Encapsulated pixel data decodes through libjpeg, OpenJPEG, CharLS, OpenJPH, and FFmpeg-class libraries, each with a long CVE history [10], deep inside the PACS or viewer. Recent crafted-file vulnerabilities in clinical software include the MicroDicom viewer remote code execution (CVE-2025-5943) [9]. Internet-exposed infrastructure compounds the risk: scanning by Greenbone Networks in 2019 reported on the order of 1.19 billion medical images reachable through unsecured PACS [11], and later internet-wide scans (Shodan-based, 2025) found thousands of exposed DICOM servers [12].

We do not argue that CDR wins a patch race. We argue the opposite: CDR is a compensating control for the trust boundary and for the systems an automated patch loop cannot reach, including unpatchable legacy and embedded devices and software locked behind recertification. Because CDR rebuilds a file from a validated canonical form rather than detecting a specific exploit, it can neutralize unknown defects, which is the property that survives a fast, capable adversary. This paper tests whether that property holds in practice, and at what cost to diagnostic fidelity and to false positives.

The contribution of this paper is two-fold. The first is the artifact (DicomLock), released as open source so any reader can run it, audit it, and reproduce every number we report. The second is the differential-evaluation methodology used to evaluate it: most commercial CDR is evaluated by assertion. We measure neutralization by re-running a labeled adversarial corpus through both a matrix of production toolkits and through CDR, apply McNemar’s paired test to the discordant cell (an empty c-set is a falsification check that came back clean on this corpus), and grow the corpus adversarially until it breaks the defense itself. When it did, we report what broke and how

we fixed it. The methodology is general to any file format with a pre-parse attack surface and a paired toolkit matrix; we instantiate it on DICOM.

2 Background and related work

A DICOM Part-10 file consists of the preamble, the four-byte DICM marker, a File Meta group encoded in Explicit VR Little Endian that names the transfer syntax, the main data set of tagged elements, and the pixel data [20, 21]. Each region is attacker-controllable, and the transfer syntax alone decides which third-party codec will decode the pixels.

CDR is not new, and we are careful not to claim novelty for the concept. Commercial DICOM CDR exists: OPSWAT MetaDefender added DICOM support in 2024 [13], and Votiro (acquired by Menlo) markets DICOM file disarm [14]. Academic work on transcoding to disarm image polyglots and to route pixels off vulnerable decoders predates this tool (the ICDR and ImSan lines of work [15, 16]), and prior work on DICOM payload detection exists (MalDicom [17]). Network and device-security vendors (for example Claroty, Cylera, Asimily, and Forescout) monitor the network and the device rather than sanitizing the file, and antivirus engines commonly skip medical imagery.

The lane this work occupies is therefore not “no one does CDR.” It is the combination of properties that the commercial and academic options do not jointly provide for a hospital that wants to inspect what it runs: open source and auditable (every check and every transformation is readable), self-hosted (no patient data leaves the building), bit-exact on native and lossless sources with an explicit and documented policy on lossy sources, and a paired evaluation that measures neutralization against real toolkits and a pinned vulnerable codec rather than asserting it.

A note on cryptography, to protect credibility. AI-assisted discovery of this kind does not break ciphers such as AES or RSA. It finds the implementation and deployment defects around encryption (memory bugs that leak keys, authentication bypasses, decryption at the point of use), and it benefits from the fact that most DICOM in transit and at rest is cleartext. The separate risk that cryptographic mathematics is broken is the quantum harvest-now-decrypt-later threat to long-lived protected health information, which this tool does not address.

3 Threat model

In scope: a hostile DICOM file presented to a parser or codec at an ingestion boundary (outside-image import, research data intake, an AI pipeline, a viewer). The attacker controls the full byte content of the file. The defender wants to admit only files that are safe to parse and that preserve diagnostic content.

The modeled attack classes are: polyglot preamble (executable or archive header in the first 128 bytes); length amplification (an element, in the main data set or in the File Meta group, declaring more bytes than the file contains); sequence-nesting bombs; pixel dimension and decompression bombs (header dimensions or a tiny encapsulated payload that forces a multi-gigabyte allocation); private-tag payloads (executables or opaque high-entropy blobs smuggled in odd-group tags); and routing through a codec with known memory-safety defects.

Out of scope, stated plainly so the contribution is precise: network and authentication defects (for example the Orthanc authentication bypass CVE-2025-0896 [8] is a server flaw, not a file-parse bug, and CDR neither can nor should claim to fix it); attacks that do not pass through the file boundary; and exploitability of any specific pinned-version crash on a current, patched system.

4 System design

DicomLock has two stages, a scanner and a CDR engine, sharing one pipeline so that what the scanner flags is what the CDR removes.

4.1 Scanner (decode-free by design)

The scanner never decodes pixels, because decoding is exactly the untrusted codec path the tool exists to avoid. It runs six checks. The preamble check matches executable and archive signatures in the first 128 bytes and reports residual non-zero preambles by entropy. The length-amplification check walks elements at the byte level, validating both the File Meta group and the main data set, and stops cleanly when the declared Explicit VR encoding does not match the bytes (so it does not fabricate a bomb on an encoding mismatch). The sequence-depth check rejects nesting beyond a safe limit. The pixel-dimension check flags header dimensions or amplification ratios that would force an implausible allocation, with a tiered response (an extreme ratio is blocked, a moderate ratio is warned). The private-tag check classifies odd-group binary values as payloads when they carry a known signature (at offset zero or padded deeper into the value) or when they are opaque and high-entropy. A codec-exposure check maps the transfer syntax to its decoder and to known CVEs, reporting exposure rather than asserting exploitability.

4.2 CDR engine

For a recoverable file, the engine rebuilds a clean canonical DICOM. It zeroes the preamble, which neutralizes polyglots independently of the signature. It transcodes compressed pixel data to a native transfer syntax by decoding once and storing uncompressed, which removes the codec attack surface without introducing new loss; the decode runs in a resource-limited subprocess so that a crash, hang, or out-of-memory in the third-party codec is contained and the file is quarantined rather than taking down the tool. It filters private tags against a default-deny vendor allowlist, keeping recognized vendor creators but stripping unknown creators and stripping any value that carries a payload, even under an allowlisted creator. It then re-scans the rebuilt file and emits it only if it is clean, quarantining otherwise. Files with no recoverable image, and allocation, length, or decompression bombs, are rejected before any decode is attempted.

Fidelity is labeled honestly. For native sources and mathematically lossless encapsulated sources, a transcode is bit-exact against the original acquisition. For lossy sources, the pixels are preserved exactly as decoded (no new loss is introduced), but the result is not claimed bit-exact against the original acquisition, because that loss already occurred in the source. Sources that cannot be decoded are quarantined, not silently passed.

5 Evaluation methodology

We built a benchmark engine that pairs with the package, imports it, and is not shipped in the wheel. For each file it records the scanner verdict; runs a matrix of production toolkits (pydicom, GDCM, and dcmstk's `dcmdump`) in resource-limited subprocesses so that a C-level fault or hang is observable as a process signal rather than swallowed by an exception; disarms the file; and then re-runs both the toolkit matrix and the scanner on the disarmed output. Allocation and decompression bombs are pre-identified and never executed raw, since that decode is the attack.

The corpus has three parts. A curated set of inert tampered fixtures exercises each attack class. An adversarial generator writes a larger set of inert, labeled, deliberately hard files (signature and

offset variants, boundary probes for length, nesting, and dimensions, payloads under allowlisted creators, chained attacks, and benign edge cases), with the explicit goal of slipping past the scanner or defeating CDR. The benign set is 30 curated files plus 575 real TCIA clinical CTs (used scan-only, since benign files do not need the crash matrix), and a separate fidelity harness adds the diverse DICOM test data bundled with pydicom and pylibjpeg, which spans many modalities and every common transfer syntax. To exercise the codec and private-tag paths that uniform chest CT data does not, and to test whether the false-positive and fidelity numbers hold across a different body region, we additionally pulled 370 real clinical files from TCIA in three further public datasets: 120 brain MR from UPENN-GBM, 150 chest radiographs from LIDC-IDRI, and 100 abdomen CT from TCGA-KIRC, and ran both the scanner and the CDR rebuild over them (`bench.diverse_check`). The chest CT corpus was sampled across LIDC-IDRI, NSCLC-Radiomics, TCGA-LUAD, and COVID-19-AR. Per-collection sources and data-use terms are listed under reference [24].

A separate harness runs the JPEG 2000 pixel stream of a file through a pinned OpenJPEG 2.3.0 build compiled with AddressSanitizer, inside a container with a memory limit, to measure whether CDR neutralizes what a real vulnerable decoder chokes on.

Metrics: detection (tampered files flagged at the expected severity), false positives (benign files blocked), neutralization (dangerous inputs that are quarantined or rebuilt clean to every toolkit and to the scanner), bit-exact fidelity, and differentiation (files DicomLock flags that the toolkits accept). We report Wilson 95% confidence intervals for proportions, a one-sided 95% upper bound (the rule of three) for the zero-event false-positive rate, and McNemar’s paired test comparing DicomLock to the toolkit matrix. All statistics are computed without external numerical dependencies.

The methodology, as a reusable protocol. The procedure above describes how we evaluate DicomLock; the structure is general. It has four moving parts: a labeled, inert adversarial corpus that targets a defined attack surface; a matrix of mature production parsers or decoders for the file format, run in resource-limited subprocesses; a CDR engine that rebuilds a clean canonical file; and a re-evaluation pass that runs the matrix on the disarmed output. The headline metric is McNemar’s paired test on the discordant cell (the matrix accepts a file the CDR flags, and the reverse); an empty c-cell is a falsification check that the CDR has no blind spot against the matrix. To prevent the corpus from being too easy on the defense, an adversarial generator targets the implementation’s weak points (boundary probes, signature variants, payloads under allowlisted private creators, chained attacks, benign edge cases) until the loop produces a CDR defeat, at which point the defeat is fixed, regression-guarded, and published. We instantiate this protocol on DICOM; no step is DICOM-specific. The natural next instantiations are PDF, Office-document, and image-container CDR, each of which has its own paired toolkit matrix and its own pre-parse attack surface. We do not demonstrate the protocol empirically on a second file format here; we argue its generalization by structure and leave the empirical demonstration to future work.

6 Results

All numbers below are from one command (`python -m bench`) plus the fidelity-at-scale harness (`python -m bench.fidelity`), reproduced 2026-05-25. Table 1 summarizes the headline measures and Table 2 the residual re-identification audit.

Detection. 80 of 80 tampered files were flagged at the expected severity (Wilson 95% CI 95.4 to 100 percent).

Table 1: Headline results. Benign files are 30 curated plus 575 real clinical CTs; an additional 370 real clinical files in three further datasets and a 103-file mixed-compression corpus also produced 0 false positives, so 0 across all 945 real clinical files used here.

Metric	Result	95% confidence
Detection (tampered flagged as expected)	80/80	95.4 to 100% (Wilson)
False positives (benign blocked)	0/605	$\leq 0.50\%$ (rule of three)
Neutralization (dangerous inputs made safe)	80/80	95.4 to 100% (Wilson)
Fidelity (native and lossless, bit-exact)	623/623	0 breaks
Differentiation (flagged, toolkits accept)	51 files	$\chi^2 = 49.0$, $p < 10^{-6}$

False positives. 0 of 605 benign files were blocked (30 curated plus 575 real CTs). With zero events, the one-sided 95% upper bound on the false-positive rate is 0.50 percent. A separate mixed-compression corpus of 103 files spanning 12 transfer syntaxes produced 0 false positives on conformant files; the 8 files given a blocking verdict were each genuinely non-conformant (no Part-10 header, truncated, or missing image dimensions). A further 370 real clinical files in three additional public datasets (120 brain MR from UPENN-GBM, 150 chest radiographs from LIDC-IDRI, and 100 abdomen CT from TCGA-KIRC) produced 0 false positives, so across all 945 real clinical files used here (575 chest CT, 100 abdomen CT, 120 brain MR, 150 chest XR; three modalities and three body regions) the scanner blocked none.

Neutralization. 80 of 80 dangerous inputs were made safe, by quarantine for the un-rebuildable classes (length, dimension, and decompression bombs) and by clean rebuild for the rest (Wilson 95% CI 95.4 to 100 percent).

Fidelity at scale. Across a diverse benign corpus plus the real CTs, 623 of 623 files with native or lossless sources were rebuilt bit-exact (575 CTs and 48 diverse files), spanning 13 transfer syntaxes (Implicit and Explicit VR Little Endian, Explicit VR Big Endian, RLE Lossless, JPEG 2000 Lossless, JPEG Lossless, JPEG-LS Lossless, and Deflated, among others). 20 of 20 lossy-source files had their pixels preserved exactly as decoded. The 370 additional brain MR, chest radiography, and abdomen CT files were each rebuilt bit-exact as well, including the JPEG Lossless and JPEG 2000 Lossless codec paths the chest CT corpus does not contain. There were 0 fidelity breaks across every modality and body region tested.

Performance. Single-threaded on commodity laptop hardware (Python 3.12, macOS arm64), the scanner processes 945 real clinical files at 254 files per second and 456 MiB per second (per-file median 2 to 10 milliseconds depending on file size, p95 4 to 21 milliseconds), with a peak resident memory of 265 MiB. The CDR rebuild adds the cost of the sandboxed codec subprocess on encapsulated transfer syntaxes: on a native chest CT corpus the disarm path runs at 174 files per second, and on a mixed-codec brain MR corpus (a small fraction JPEG Lossless and JPEG 2000 Lossless) it runs at 63 files per second. A typical 10,000-image PACS day clears scanning in under a minute on one core; the workload is embarrassingly parallel, so a real deployment scales linearly. Reproduced with `python -m bench.perf --include-disarm`.

Differentiation. On the 63 tampered files the toolkits actually executed (excluding pre-identified bombs, which are never run raw), DicomLock flagged 51 files that every toolkit (pydicom, GDCM,

dcmTk) accepted as valid, and 0 files that DicomLock passed as clean were rejected by any toolkit. McNemar’s paired test gives $\chi^2 = 49.0$, $p < 10^{-6}$. The empty discordant cell (no DicomLock blind spots) is itself a falsification check that came back clean.

Pinned vulnerable codec. A fuzzer-found malformed JPEG 2000 file drives the pinned Open-JPEG 2.3.0 + AddressSanitizer build to a fault. CDR neutralizes the malicious DICOM carrier by quarantine. A clean image disarms bit-exact. One nuance matters for deployment and we keep it in the paper: the static scanner does not flag that file, because its header declares a small image; only the sandboxed CDR decode catches the codec-level bomb. This argues for running disarm, not scan-only, at a hostile boundary. The fault we reproduced is in the denial-of-service and allocation class. We do not claim a reproduced memory-corruption (heap-overflow or use-after-free) exploit.

Falsification as a worked example. Growing the corpus adversarially and probing the implementation at its boundaries surfaced five real defects that a homogeneous corpus had hidden. The most serious was a CDR escape: a payload hidden under an allowlisted vendor creator survived disarm, because the private-tag override only matched a listed signature at the first byte. A control (the same payloads under an unknown creator) confirmed the allowlist itself was correct, which isolated the defect. We fixed it with a shared classifier that strips a value carrying a signature anywhere in a leading window, or any opaque high-entropy value, even under a known creator. The other four were missed installer and archive polyglot signatures (OLE compound files, CAB, Zstandard), an unvalidated length bomb in the File Meta group, a moderate-amplification decompression bomb that previously produced only a codec-exposure warning, and a private payload below an old size floor. All five were fixed with zero new false positives. Fix thresholds were grounded in measured real vendor data (61 files, 212 private binary tags, median size 4 bytes, maximum entropy 3.75 out of 8), so the high-entropy strip does not fire on legitimate metadata. One residual is documented rather than fixed: a low-entropy, signature-less blob under an allowlisted creator is preserved by design, because it is indistinguishable from real vendor data. We also prototyped and then removed an image-media polyglot tier, because it false-flagged a standard test file that carries a benign TIFF header in its preamble.

Privacy auditing. As an optional, separate capability, the tool computes an ordinal re-identification-risk score (0 to 100) over four channels (residual structured identifiers, identifiers in free text and private tags, burned-in pixel text, and facial-geometry reconstructability) [18, 19], ranking a clean file at 0 (MINIMAL), a file with residual identifiers at 45 (MODERATE), and a fully identified file at 100 (HIGH). It is a triage score, not a certification of de-identification.

Applied to 1,045 public “de-identified” files across five datasets, four modalities, and four body regions (575 chest CT, 100 abdomen CT from TCGA-KIRC, 120 brain MR from UPENN-GBM, 150 chest radiographs from LIDC-IDRI, and 100 mammograms from CBIS-DDSM), the audit finds substantial pixel-domain residual risk that no tag anonymizer can fix, and the risk varies systematically by modality and anatomical region. Facial-geometry risk fires on 96.7 percent of the head MR (the Mayo concern [18, 19]) and below one percent on every non-head dataset (0.3 percent on chest CT, 0.0 percent on chest XR, abdomen CT, and mammography), sanity-confirming that the channel is anatomically gated rather than spuriously high. Burned-in pixel text fires on 89.3 percent of the chest radiographs, 52.0 percent of the mammograms, 22.5 percent of the brain MR, 17.0 percent of the abdomen CT, and 8.0 percent of the chest CT. The factor of two difference in burned-in rate between abdomen CT and chest CT, on otherwise comparable scanners and the same modality, is a finding in itself: pixel-domain re-identification risk is not a property of modality

alone but of scanner protocol per body region. Across modalities the rate spans more than an order of magnitude (8 to 89 percent), with mammography at 52 percent in between, so no body-region-level policy alone is sufficient either. A paired comparison against a standard tag anonymizer (dicognito 0.19) on 60 of the brain MR confirmed the gap: the anonymizer changed every direct identifier (120 of 120, tag linkage broken) but left the pixel data byte-identical (60 of 60), so the pixel-domain channels reported above are provably unchanged by current tag-based anonymization. The tag-domain channels also contribute to the ordinal score because TCIA pseudonymizes rather than empties tags, and we report that floor honestly rather than treat it as undetected direct PHI. The harnesses for both results (`bench.reid_audit` and `bench.reid_vs_anonymizer`) ship in the released artifact.

Table 2: Residual re-identification risk by dataset (public, already “de-identified” imaging). The facial-geometry channel is anatomically gated; burned-in pixel text varies both by body region within a single modality (chest CT 8.0% vs abdomen CT 17.0%) and across modalities (8.0% on chest CT to 89.3% on chest radiography, with mammography at 52.0% in between). No real participant was re-identified; the facial channel is a metadata-derived structural-exposure metric, not a face match.

Dataset (modality, region)	Source	Facial geometry	Burned-in text
Chest CT	LIDC-IDRI et al.	0.3%	8.0%
Abdomen CT	TCGA-KIRC	0.0%	17.0%
Brain MR	UPENN-GBM	96.7%	22.5%
Chest radiography	LIDC-IDRI	0.0%	89.3%
Mammography	CBIS-DDSM	0.0%	52.0%

7 Discussion

The results support the narrow claim we set out to test: rebuilding a DICOM file from a validated canonical form neutralizes the modeled file and codec attack classes while preserving native and lossless images bit-exact, with a false-positive rate whose 95% upper bound is below one percent on 605 benign files. The differentiation result quantifies the gap the tool fills: three mature toolkits accept dozens of weaponized files as valid that DicomLock flags, and the tool introduces no blind spot relative to those toolkits on this corpus.

Two points of intellectual honesty shape how the result should be read. First, the scan-only path does not catch a codec-level bomb whose header looks benign; only the sandboxed disarm does. The practical recommendation is therefore to disarm at a hostile boundary, not merely to scan. Second, the value of the falsification round is not the headline numbers but the defect it found in our own defense. A benchmark that only ever reports success on an easy corpus is not evidence; we treat a zero-failure result as something to distrust until the corpus is hard enough to break the tool, and we ship the generator that does the breaking.

A note on what the paper actually contributes. The DicomLock artifact is one of several CDR systems for DICOM; commercial DICOM CDR exists and academic transcoding-based image sanitization predates this work. The artifact’s value lies in being open source, self-hosted, auditable, and PACS-depth, not in the concept of file-level CDR. The contribution that does not have a clear precedent is the evaluation methodology in Section 5: a paired matrix-versus-CDR loop with McNemar on the discordant cell, an adversarial generator that keeps growing the corpus until it produces a defeat, and the defeat published as part of the paper rather than hidden. We expect the same

loop, applied to PDF or to an image-container format, to surface the same kind of self-discovered defect we report here for DICOM.

8 Limitations and threats to validity

The threat is demonstrated through proofs of concept and CVEs rather than documented in the wild through a weaponized DICOM file, so urgency is partly anticipatory. No current standard mandates file sanitization either, so there is no direct procurement trigger; the nearest policy anchors are NIST SP 1800-24 for securing PACS [22] and the FDA premarket cybersecurity guidance for device input-robustness [23]. A crash in a pinned old version is not the same as exploitability on a current system; we measure crash neutralization, not end-to-end exploit prevention. The reproduced codec fault is denial-of-service class, not memory corruption. Bit-exact fidelity applies to native and lossless sources; lossy and proprietary-only inputs are quarantined or preserved-as-decoded, not recovered bit-exact. The benchmark corpus, though adversarial and diverse, is finite and under-counts the true attack surface. The codec-CVE and vendor-allowlist data files are illustrative seeds and should be verified against authoritative sources before any vendor-facing claim. Commercial and academic CDR exist, so the contribution is openness, auditability, the paired methodology, and PACS-depth, not the concept of CDR. The claim that the methodology generalizes beyond DICOM is structural, not empirical: each step is independent of any DICOM-specific detail and other file formats have analogous toolkit matrices and pre-parse attack surfaces, but we have not instantiated the loop on a second format in this work.

9 Responsible disclosure and ethics

Every shipped artifact is inert: attack fixtures use real magic bytes or headers followed by zero padding, with no working malware. Any new defect surfaced in a third-party component goes through coordinated disclosure to the maintainer before publication. The evaluation uses only public, de-identified imaging, so it requires no protected health information and no IRB review; dataset licenses (TCIA [24]) are re-confirmed before release.

10 Availability

DicomLock is open source under Apache-2.0 at <https://github.com/vthakore23/dicomlock> and installable with `pip install dicomlock`. The benchmark engine, the adversarial generator, the fidelity-at-scale harness, and the pinned-codec Dockerfiles are released for reproducibility. The recommended deployment is self-hosted, with no network egress, so that no patient data leaves the building.

References

- [1] Anthropic. Project Glasswing: Securing critical software for the AI era. 2026. <https://www.anthropic.com/glasswing>
- [2] Anthropic. Project Glasswing: An initial update. 2026. <https://www.anthropic.com/research/glasswing-initial-update>

- [3] Anthropic. Claude Mythos Preview. 2026. <https://red.anthropic.com/2026/mythos-preview/> (Reported: more than 10,000 high or critical vulnerabilities across systemically important software; approximately 50 partners; CVE-2026-4747, a 17-year-old FreeBSD NFS remote code execution, found and exploited via Mythos, with the discovery-versus-recall debate noted in the text; 271 Firefox vulnerabilities with working exploits for 181.)
- [4] Health care’s biggest cybersecurity vulnerability is structural. STAT, 17 April 2026. <https://www.statnews.com/2026/04/17/health-care-cybersecurity-ransomware-project-glasswing/>
- [5] Health-ISAC. How Claude Mythos could impact healthcare cybersecurity. Reported by TechTarget, 2026. (Health-ISAC: “Should the rollout of Claude Mythos follow a similar trajectory, it could jeopardize health sector security.”)
- [6] CVE-2019-11687. National Vulnerability Database. (Executable header in the DICOM preamble.)
- [7] Praetorian. ELFDICOM: a polyglot DICOM that is also a Linux ELF executable. April 2025.
- [8] Orthanc authentication bypass. CVE-2025-0896 (CVSS 9.8). National Vulnerability Database. (A server authentication flaw, cited as the out-of-scope network class.)
- [9] MicroDicom DICOM Viewer out-of-bounds write. CVE-2025-5943. National Vulnerability Database. (Crafted-file remote code execution, in scope for file CDR.)
- [10] Representative codec memory-safety CVEs routed into by DICOM transfer syntaxes (from the project’s auditable map, `scanner/data/dicom_codec_cve.json`): OpenJPEG CVE-2020-27814, CVE-2018-5785; libjpeg-turbo CVE-2018-19664, CVE-2018-20330; zlib CVE-2022-37434; FFmpeg-class CVE-2016-10190. Listed as a maintained seed to verify against NVD and CISA, not as proof of exploitability.
- [11] Greenbone Networks. Unsecured medical-imaging systems exposing on the order of 1.19 billion images. 2019 (reported November 2019; coverage in TechCrunch and the HIPAA Journal, January 2020).
- [12] Shodan-based internet-wide scanning of exposed DICOM servers (on the order of 3,627 servers), 2025.
- [13] OPSWAT. OPSWAT Deep CDR now supports DICOM file format. June 2024. <https://www.opswat.com/blog/opswat-deep-cdr-now-supports-dicom-file-format>
- [14] Votiro (acquired by Menlo Security, 2025). DICOM file disarm (vendor product).
- [15] ICDR: image Content Disarm and Reconstruction. arXiv:2307.14057. (Open image CDR; names DICOM as future work; code at <https://github.com/ArielCyber/ICDR>.)
- [16] ImSan: image sanitization against polyglots. arXiv:2407.01529.
- [17] MalDicom: malware detection in DICOM files. arXiv:2312.00483. (Detection, not CDR.)
- [18] Schwarz CG, et al. Identification of anonymous MRI research participants with face-recognition software. N Engl J Med 2019;381:1684-1686. doi:10.1056/NEJMc1908881. (Reported an 83 percent match rate from cranial MRI.)
- [19] Measuring the potential risk of re-identification of imaging research participants from open-source automated face recognition software. Mayo Clinic and Carnegie Mellon University, 2025. PMC11714269. (Commercial face recognition re-identified brain MRI, CT, and PET at up to 98 percent, an escalation of the 2019 result above.)
- [20] NEMA. DICOM PS3.10, Media Storage and File Format for Media Interchange. DICOM Standard, 2026.
- [21] NEMA. DICOM PS3.15, Security and System Management Profiles. DICOM Standard, 2026.

- [22] NIST. SP 1800-24, Securing Picture Archiving and Communication System (PACS): Cybersecurity for the Healthcare Sector. December 2020. <https://www.nccoe.nist.gov/publication/1800-24/>
- [23] U.S. Food and Drug Administration. Cybersecurity in Medical Devices: Quality System Considerations and Content of Premarket Submissions. Final guidance, 27 September 2023 (Select Updates finalized June 2025).
- [24] Clark K, Vendt B, Smith K, et al. The Cancer Imaging Archive (TCIA): maintaining and operating a public information repository. *Journal of Digital Imaging* 2013;26(6):1045-1057. (Source of the real clinical CT, MR, and radiography used for the false-positive and fidelity evaluation; per-collection citations and data-use terms are confirmed before artifact release.) Per-collection sources used in this work: LIDC-IDRI (<https://www.cancerimagingarchive.net/collection/lidc-idri/>), NSCLC-Radiomics (<https://www.cancerimagingarchive.net/collection/nsclc-radiomics/>), TCGA-LUAD (<https://www.cancerimagingarchive.net/collection/tcga-luad/>), COVID-19-AR (<https://www.cancerimagingarchive.net/collection/covid-19-ar/>), UPENN-GBM (<https://www.cancerimagingarchive.net/collection/upenn-gbm/>), and TCGA-KIRC (<https://www.cancerimagingarchive.net/collection/tcga-kirc/>).

A JAMIA submission format

The free-form abstract above is the arXiv version. For a JAMIA submission, replace it with the structured abstract below and insert the Statement of Significance before Section 1. JAMIA expects section names “Background and significance,” “Objective,” “Materials and Methods,” “Results,” “Discussion,” and “Conclusion.” The current section names map directly: Sections 1 and 2 become “Background and significance,” Section 3 becomes the “Objective” paragraph (a sentence or two), Sections 4 and 5 become “Materials and Methods,” Section 6 stays “Results,” Sections 7 and 8 fold into “Discussion,” and Section 9 plus a one-paragraph summary becomes “Conclusion.”

A.1 Structured abstract (target about 250 words)

Objective. Test whether decoding-free scanning paired with Content Disarm and Reconstruction (CDR) neutralizes the modeled file and codec attack classes on DICOM medical-image files while preserving diagnostic fidelity, and quantify the residual re-identification risk that current tag anonymization leaves in the pixel domain.

Materials and Methods. We built DicomLock, an open-source self-hosted scanner and CDR engine (`pip install dicomlock`, Apache-2.0). A benchmark engine runs a labeled corpus of inert tampered files through three production toolkits (pydicom, GDCM, dcmstk) and through CDR, and grows the corpus adversarially. The false-positive and fidelity evaluation uses 945 real clinical files across four public TCIA collections (LIDC-IDRI / NSCLC-Radiomics / TCGA-LUAD / COVID-19-AR chest CT, TCGA-KIRC abdomen CT, UPENN-GBM brain MR, LIDC-IDRI chest radiography) spanning 3 modalities and 3 body regions. A residual re-identification audit pairs DicomLock’s ordinal score with a standard tag anonymizer (dicognito 0.19) on those files plus 100 mammograms from CBIS-DDSM, for a 1,045-file re-identification audit corpus spanning 4 modalities and 4 body regions.

Results. 80 of 80 tampered files detected, 80 of 80 neutralized, 0 false positives across 945 real benign files plus 30 curated (one-sided 95% upper bound 0.50%), CDR rebuilds bit-exact on every native and lossless source across 13 transfer syntaxes (623 of 623). DicomLock flagged 51 files every reference toolkit accepted as valid, with 0 discordant cases against any toolkit (McNemar $\chi^2 = 49.0$, $p < 10^{-6}$). The audit found pixel-domain re-identification risk that tag anonymization

cannot remove: facial-geometry features on 96.7% of brain MR, and burned-in pixel text on 89.3% of chest radiographs, 52.0% of mammograms, 22.5% of brain MR, 17.0% of abdomen CT, and 8.0% of chest CT (scanner protocol per body region, not modality alone).

Discussion. CDR is a compensating control for the systems an automated patch loop cannot reach, not a replacement for patching, and tag-only anonymization is not re-identification safety.

Conclusion. Decoding-free scanning plus sandboxed CDR is reproducibly safe on real clinical imaging and is an addressable defense for systems that are slow to patch.

A.2 Statement of Significance (target about 250 words)

Problem. Hospitals run exactly the software AI-assisted vulnerability discovery now targets (DICOM toolkits and image and video codecs), and they patch slowly because of device recertification and legacy or embedded equipment. The health sector was explicitly omitted from the early-access protection group for one such system in 2026, an omission Health-ISAC publicly warned could jeopardize health-sector security. At the same time, current tag-based de-identification on public “de-identified” medical imaging leaves facial geometry and burned-in pixel text in place, channels that recent work has shown can re-identify research participants with up to 98 percent accuracy.

What is already known. Commercial DICOM CDR products exist (OPSWAT MetaDefender added DICOM support in 2024, Votiro markets DICOM file disarm), and academic transcoding-based image sanitizers predate this work. Healthcare device-security vendors monitor network and device, not the file. No current standard mandates DICOM file sanitization.

What this study adds. A reproducible, open-source artifact that scans and disarms DICOM files at PACS depth, evaluated as a paired methodology rather than asserted, with the benchmark engine, adversarial generator, fidelity-at-scale harness, and pinned vulnerable codec all released. An empirical residual re-identification finding across 1,045 public files showing that the pixel-domain risk standard tag anonymization cannot fix is large and varies by both anatomical region within a modality and across modalities. A self-discovered and transparently fixed defect in our own CDR is reported as a worked example of the falsification methodology.

A.3 Data and code availability

Code, benchmark engine, adversarial generator, fidelity harness, and pinned-codec Dockerfiles are released under Apache-2.0 at <https://github.com/vthakore23/dicomlock> and on PyPI (`pip install dicomlock`). The clinical evaluation uses public TCIA collections listed under reference [24]; no protected health information is shipped or used. `REPRODUCE.md` maps each headline number in this paper to its exact command.

A.4 Author contributions, funding, and competing interests

To be completed at submission. Author contributions per CRediT taxonomy. No external funding. No competing interests to declare.